

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Малахов Алексей

БР1.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Опираясь на документ, сформированный в рамках ДЗ4, реализовать разделение монолитного бэкенд-приложения на микросервисы.

Ход работы

Монолитное приложение из ЛР1 декомпозировано на 6 микросервисов по принципу database-per-service. Каждый сервис реализован на TypeScript + Express (routing-controllers, TypeORM) и имеет собственную базу данных PostgreSQL.

Сервис	Ответственность	Порт	БД
Auth Service	Регистрация, логин, JWT	3001	5433
User Service	Профили, роли пользователей	3002	5434
Property Service	Объекты недвижимости, фото, избранное	3003	5435
Rental Service	Аренды, транзакции	3004	5436
Messaging Service	Диалоги и сообщения	3005	5437
Review Service	Отзывы об арендодателях	3006	5438

Дополнительно развёрнут **API Gateway** (порт 8000) — reverse-проxy, маршрутизирующий запросы по префиксу пути к соответствующему сервису.

Межсервисные вызовы реализованы через HTTP по маршрутам `/internal/...`, защищённым заголовком `X-Service-Token`. TypeORM-связи между сервисами отсутствуют: внешние идентификаторы хранятся как обычные колонки, данные смежных сервисов запрашиваются через HTTP. JWT-токен содержит роли пользователя (`{ user: { id, roles } }`), что позволяет сервисам проверять права без дополнительных запросов.

Часть синхронных взаимодействий заменена на **асинхронные** через RabbitMQ — реализована цепочка из трёх сервисов и двух очередей:

```
rental-service → [rental_events] → property-service → [property_events]
                → review-service
```

Rental Service публикует событие смены статуса аренды в очередь `rental_events`. Property Service обрабатывает его, обновляет статус объекта в своей БД и публикует событие в очередь `property_events`. Review Service подписывается на `property_events` и фиксирует, что отзыв об арендодателе стал доступен для написания.

Всё окружение описано в `docker-compose.yml`: 6 инстансов PostgreSQL, RabbitMQ (с management-плагином на порту 15672) и все сервисы с Dockerfile на основе многоэтапной сборки.

Трудности и решения

При декомпозиции возникла задача согласования идентификаторов пользователей между Auth Service и User Service. Решение: Auth Service генерирует id (auto-increment в таблице auth_users) и передаёт его в User Service при регистрации через POST /internal/users; User Service хранит запись с @PrimaryColumn (ID задаётся извне).

Вторая сложность — определение ownerId аренды без лишнего HTTP-вызова при каждой проверке прав. Решение: при создании аренды ownerId сохраняется прямо в таблице rentals, хотя в монолите этого поля не было.

Вывод

Монолит разделён на 6 независимых микросервисов с изолированными базами данных и единой точкой входа через API Gateway. Реализовано как синхронное межсервисное взаимодействие через REST, так и асинхронное через RabbitMQ. Все сервисы контейнеризированы и запускаются одной командой `docker compose up`.